

Blade: el motor de plantillas de Laravel

Directivas, layouts, componentes y su combinación con Tailwind CSS

Laravel 13 · Tailwind v4 · Proyecto acumulativo eventos-app · Duración: 6 horas

M. Sc. Huáscar Fedor Gonzales Guzmán · Carrera de Ingeniería Informática · UATF

Objetivos de aprendizaje

Al finalizar este laboratorio, usted será capaz de:

- Explicar cómo Blade compila una plantilla a PHP plano y usar la sintaxis de salida con y sin escape de forma segura.
- Construir vistas dinámicas con directivas de control (@if, @foreach, @forelse) y la variable \$loop.
- Aplicar clases de Tailwind de manera condicional con la directiva @class.
- Diseñar un layout reutilizable con componentes (<x-layout>, slots) y contrastarlo con la herencia clásica (@extends / @yield).
- Crear componentes Blade anónimos y de clase usando @props, slots con nombre y el objeto \$attributes.
- Reutilizar fragmentos con @include, inyectar assets por página con @push / @stack y poblar formularios con @checked, @selected y @disabled.

ANTES DE EMPEZAR

Esta guía continúa el proyecto acumulativo eventos-app que usted inició en la GL03 (Laravel 13 + Tailwind v4, sin base de datos todavía). Se asume que ya tiene el proyecto corriendo con php artisan serve y npm run dev activos, y su repositorio INF560-2026-02-Apellido sincronizado con GitHub.

01 ¿Qué es Blade y cómo funciona?

Blade es el motor de plantillas que trae Laravel. Un archivo de vista con extensión `.blade.php` no se envía tal cual al navegador: Laravel lo **compila a código PHP plano** la primera vez que se solicita y guarda el resultado en `storage/framework/views/`. En las siguientes peticiones se sirve la versión compilada y cacheada, así que Blade no impone penalización de rendimiento: en producción es PHP puro.

DEFINICIÓN

Una **directiva** es una instrucción de Blade que empieza con @ (por ejemplo @if, @foreach). Blade la traduce a su equivalente en PHP al compilar la vista.

Sintaxis de salida

Blade ofrece tres formas de escribir en la plantilla. La diferencia entre las dos primeras es de seguridad:

```
resources/views/demo.blade.php

{{-- Imprime con escape: convierte < > & " en entidades HTML --}}
{{ $titulo }}

{{-- Imprime SIN escape: renderiza el HTML tal cual --}}
{!! $htmlConfiable !!}

{{-- Esto es un comentario de Blade: no aparece en el HTML final --}}
```

[Copiar](#)

La forma con doble llave `{{ $var }}` pasa el valor por `htmlspecialchars()`, neutralizando cualquier etiqueta que venga en los datos. Es la que usted debe usar el 99 % del tiempo.

ADVERTENCIA

Use `{!! !!}` únicamente con contenido en el que confía plenamente (por ejemplo, HTML que usted mismo generó). Imprimir datos del usuario sin escape abre la puerta a **inyección de scripts (XSS)**.

02 Ejercicio 1 — Directivas de control y la variable \$loop

Construiremos la cartelera de eventos a partir de un arreglo definido en el controlador. El objetivo es dominar las directivas de condición y de bucle, y usar `$loop` y `@class` para dar estilo dinámico con Tailwind.

Paso 1 · Ruta y controlador

Genere el controlador con `php artisan make:controller EventoController` y registre la ruta:

routes/web.php

Copiar

```
use App\Http\Controllers\EventoController;

Route::get('/eventos', [EventoController::class, 'index'])->name('eventos.index');
```

app/Http/Controllers/EventoController.php

Copiar

```
<?php

namespace App\Http\Controllers;

class EventoController extends Controller
{
    public function index()
    {
        $eventos = [
            ['titulo' => 'Hackathon UATF 2026', 'fecha' => '2026-05-10', 'lugar' => 'Campus Central', 'categoria' => 'Tecnología', 'destacado' => true, 'cupos' => 120],
            ['titulo' => 'Feria del Libro', 'fecha' => '2026-05-18', 'lugar' => 'Paraninfo', 'categoria' => 'Cultura', 'destacado' => false, 'cupos' => 0],
            ['titulo' => 'Torneo de Fútbol', 'fecha' => '2026-06-02', 'lugar' => 'Coliseo', 'categoria' => 'Deporte', 'destacado' => false, 'cupos' => 32],
        ];

        return view('eventos.index', compact('eventos'));
    }
}
```

NOTA

`compact('eventos')` es equivalente a `['eventos' => $eventos]`. Dentro de la vista, la clave del arreglo se convierte en el nombre de la variable disponible: `$eventos`.

Paso 2 · La vista con @forelse, @if y @class

Cree `resources/views/eventos/index.blade.php`. Por ahora es una página completa; en el Ejercicio 2 extraeremos el layout.

resources/views/eventos/index.blade.php

Copiar

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Eventos</title>
  @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body class="bg-slate-50 text-slate-800">
  <main class="mx-auto max-w-3xl p-6">
    <h1 class="mb-6 text-2xl font-bold">Cartelera de eventos</h1>

    @forelse ($eventos as $evento)
      <article @class([
        'mb-4 rounded-xl border p-5 transition',
        'border-slate-200 bg-white' => ! $evento['destacado'],
        'border-blue-500 bg-blue-50 shadow' => $evento['destacado'],
      ])>
        <div class="flex items-center justify-between">
          <h2 class="text-lg font-semibold">{{ $evento['titulo'] }}</h2>

          @if ($loop->first)
            <span class="font-mono text-xs text-blue-600">PROXIMO</span>
          @endif
        </div>

        <p class="text-sm text-slate-500">
          {{ $evento['fecha'] }} – {{ $evento['lugar'] }}
        </p>

        @if ($evento['cupos'] > 0)
          <p class="text-sm text-emerald-600">{{ $evento['cupos'] }} cupos
disponibles</p>
        @else
          <p class="text-sm text-rose-600">Agotado</p>
        @endif
      </article>
    @empty
      <p class="text-slate-500">No hay eventos programados.</p>
    @endforelse
  </main>
</body>
</html>

```

¿Qué hace cada pieza?

La directiva `@forelse` recorre el arreglo, pero si está vacío ejecuta el bloque `@empty`. Es la fusión de un `@foreach` con un `@if (empty(...))`; ideal para listados que pueden no traer resultados.

La directiva `@class` recibe un arreglo: los elementos con clave *numérica* se aplican siempre, y los que tienen clave de texto se aplican solo si su valor es verdadero. Así elegimos el borde y el fondo según `$evento['destacado']` sin escribir un ternario ilegible dentro de `class`.

La variable \$loop

Dentro de cualquier `@foreach` / `@forelse`, Blade expone la variable `$loop` con información del recorrido:

Propiedad	Qué contiene
<code>\$loop->index</code>	Índice base 0 de la iteración actual.
<code>\$loop->iteration</code>	Número de iteración base 1.
<code>\$loop->first</code>	true en la primera vuelta.
<code>\$loop->last</code>	true en la última vuelta.
<code>\$loop->count</code>	Total de elementos.
<code>\$loop->even / ->odd</code>	true en iteraciones pares / impares (útil para filas alternas).

EJEMPLO

En bucles anidados, `$loop->parent` da acceso a la variable `$loop` del bucle exterior. Por ejemplo: `$loop->parent->index`.

03 Ejercicio 2 — Layouts reutilizables

Repetir el `<head>`, la barra de navegación y el pie en cada vista es insostenible. Blade ofrece dos mecanismos para definir una plantilla base: el enfoque moderno con componentes (recomendado) y la herencia clásica. Trabajaremos el primero y luego lo compararemos.

Paso 1 · El componente de layout

Cree `resources/views/components/layout.blade.php`. Todo lo que la página coloque "dentro" del componente llegará en la variable `$slot`.

`resources/views/components/layout.blade.php`

Copiar

```
@props(['title' => 'eventos-app'])

<!DOCTYPE html>
<html lang="es" class="h-full">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{{ $title }} – eventos-app</title>
  @vite(['resources/css/app.css', 'resources/js/app.js'])
  @stack('estilos')
</head>
<body class="min-h-full bg-slate-50 text-slate-800 antialiased">
  <x-navbar />

  <main class="mx-auto max-w-3xl p-6">
    {{ $slot }}
  </main>

  <x-footer />

  @stack('scripts')
</body>
</html>
```

Paso 2 · Navbar y footer como componentes

resources/views/components/navbar.blade.php

Copiar

```
<header class="border-b border-slate-200 bg-white">
  <nav class="mx-auto flex max-w-3xl items-center justify-between p-4">
    <a href="{{ route('eventos.index') }}" class="font-bold text-slate-800">eventos-
app</a>
    <ul class="flex gap-6 text-sm">
      <li><a href="{{ route('eventos.index') }}" class="text-slate-600 hover:text-blue-
600">Eventos</a></li>
      <li><a href="#" class="text-slate-600 hover:text-blue-600">Acerca de</a></li>
    </ul>
  </nav>
</header>
```

resources/views/components/footer.blade.php

Copiar

```
<footer class="border-t border-slate-200 bg-white">
  <div class="mx-auto max-w-3xl p-4 text-center text-sm text-slate-500">
    © {{ date('Y') }} eventos-app · INF560 UATF
  </div>
</footer>
```

Paso 3 · Usar el layout en la cartelera

Ahora la vista del Ejercicio 1 se reduce a su contenido propio; el resto vive en el layout:

resources/views/eventos/index.blade.php

Copiar

```
<x-layout title="Eventos">
  <h1 class="mb-6 text-2xl font-bold">Cartelera de eventos</h1>

  @forelse ($eventos as $evento)
    {{-- ... el <article> del Ejercicio 1 ... --}}
  @empty
    <p class="text-slate-500">No hay eventos programados.</p>
  @endforelse
</x-layout>
```

NOTA

El atributo title="Eventos" llega al componente porque lo declaramos en @props. Los @props definen qué datos espera el componente y con qué valor por defecto.

Alternativa clásica: @extends / @yield

Encontrará mucho código heredado con este otro patrón. Conviene reconocerlo:

resources/views/layouts/app.blade.php

Copiar

```
<!DOCTYPE html>
<html lang="es">
<head>
  <title>@yield('title', 'eventos-app')</title>
  @vite(['resources/css/app.css', 'resources/js/app.js'])
</head>
<body>
  @include('partials.navbar')
  <main class="mx-auto max-w-3xl p-6">
    @yield('contenido')
  </main>
</body>
</html>
```

resources/views/eventos/index-clasico.blade.php

Copiar

```
@extends('layouts.app')

@section('title', 'Eventos')

@section('contenido')
    <h1 class="mb-6 text-2xl font-bold">Cartelera de eventos</h1>
    {{-- ... --}}
@endsection
```

Componente <x-layout>	Herencia @extends
El contenido se pasa como slot y atributos.	El contenido se declara en @section por nombre.
Reutilizable como cualquier componente; admite varios slots.	Una vista extiende un solo layout a la vez.
Enfoque recomendado en Laravel moderno.	Sigue siendo válido; común en proyectos antiguos.

04 Ejercicio 3 — Componentes con props, slots y atributos

Vamos a encapsular la tarjeta de evento en un componente reutilizable <x-evento-card>. Veremos las dos variantes de componente y el objeto \$attributes, que es lo que permite combinar bien un componente con Tailwind.

Componente anónimo

Un **componente anónimo** es solo un archivo Blade en `resources/views/components/`, sin clase PHP. Es la opción por defecto cuando el componente no necesita lógica.

resources/views/components/evento-card.blade.php

Copiar

```
@props([
    'titulo',
    'fecha',
    'lugar',
    'destacado' => false,
])

<article {{ $attributes->class([
    'rounded-xl border p-5 transition',
    'border-slate-200 bg-white' => ! $destacado,
    'border-blue-500 bg-blue-50 shadow' => $destacado,
]) }}>
    <div class="flex items-start justify-between">
        <div>
            <h2 class="text-lg font-semibold text-slate-800">{{ $titulo }}</h2>
            <p class="text-sm text-slate-500">{{ $fecha }} – {{ $lugar }}</p>
        </div>

        {{ $badge ?? '' }} {{-- slot con nombre opcional --}}
    </div>

    @isset($footer)
        <div class="mt-4 border-t border-slate-100 pt-3">
            {{ $footer }}
        </div>
    @endisset
</article>
```

```

    </div>
    @endisset
</article>

```

Usarlo con slots con nombre y atributos extra

```

resources/views/eventos/index.blade.php
Copiar

<x-layout title="Eventos">
    <h1 class="mb-6 text-2xl font-bold">Cartelera de eventos</h1>

    @foreach ($eventos as $evento)
        <x-evento-card
            :titulo="$evento['titulo']"
            :fecha="$evento['fecha']"
            :lugar="$evento['lugar']"
            :destacado="$evento['destacado']"
            class="mb-4"
        >
            <x-slot:badge>
                <x-badge :categoria="$evento['categoria']" />
            </x-slot:badge>

            <x-slot:footer>
                <a href="#" class="text-sm font-medium text-blue-600 hover:underline">Ver
detalle →</a>
            </x-slot:footer>
        </x-evento-card>
    @endforeach
</x-layout>

```

DEFINICIÓN

El `:` antes de un atributo (`:titulo="..."`) indica que el valor es una **expresión PHP**, no un texto literal. Sin los dos puntos, `titulo="$evento['titulo']"` pasaría la cadena tal cual.

El punto clave es `$attributes`. Cuando la vista escribe `class="mb-4"` sobre la etiqueta `<x-evento-card>`, ese atributo no se pierde: llega al componente dentro de `$attributes`. Al invocar `$attributes->class(...)` fusionamos las clases del componente con las que vengan de fuera, en vez de sobrescribirlas. Si necesita fusionar otros atributos, use `$attributes->merge(['class' => '...'])`.

Componente de clase

Cuando hay **lógica** (calcular un color según la categoría, formatear una fecha, consultar algo), conviene un **componente de clase**. Genérelolo con:

```

terminal
Copiar

php artisan make:component Badge

```

Esto crea `app/View/Components/Badge.php` y `resources/views/components/badge.blade.php`. La lógica va en el constructor; las propiedades públicas quedan disponibles en la vista.

```

app/View/Components/Badge.php
Copiar

<?php

namespace App\View\Components;

use Illuminate\View\Component;

```



```
use Illuminate\Contracts\View\View;

class Badge extends Component
{
    public string $color;

    public function __construct(public string $categoria)
    {
        $this->color = match ($categoria) {
            'Tecnología' => 'bg-blue-100 text-blue-700',
            'Cultura'    => 'bg-purple-100 text-purple-700',
            'Deporte'    => 'bg-emerald-100 text-emerald-700',
            default      => 'bg-slate-100 text-slate-600',
        };
    }

    public function render(): View
    {
        return view('components.badge');
    }
}
```

resources/views/components/badge.blade.php

Copiar

```
<span {{ $attributes->class(['rounded-full px-3 py-1 text-xs font-medium', $color]) }}>
    {{ $categoria }}
</span>
```

NOTA

¿Anónimo o de clase? Regla práctica: si el componente solo maqueta HTML con los datos que recibe, **anónimo**. Si necesita calcular o transformar algo antes de mostrarlo, **de clase**.

05 Ejercicio 4 — Includes, stacks y directivas de formulario

@include vs. componente

La directiva `@include('partials.filtros')` inserta otra vista en el punto actual. A diferencia de un componente, el parcial **comparte todas las variables** de la vista que lo incluye (no hay aislamiento). Úselo para fragmentos ligados al contexto; use componentes cuando quiera una pieza autónoma y reutilizable.

Directiva	Qué hace
<code>@include('vista')</code>	Inserta la vista y le hereda las variables actuales.
<code>@includeIf('vista')</code>	La incluye solo si el archivo existe (no falla si no).
<code>@includeWhen(\$cond, 'vista')</code>	La incluye solo si la condición es verdadera.
<code>@each('v', \$items, 'item')</code>	Renderiza una vista por cada elemento de la colección.

Directivas de formulario

Blade trae atajos que evitan ternarios dentro de los atributos de un formulario. Cree el parcial de filtros (aún sin base de datos, solo lectura de la URL con `request()`):

resources/views/partials/filtros.blade.php

Copiar

```

<form method="GET" action="{{ route('eventos.index') }}" class="mb-6 flex flex-wrap items-center gap-3">
  <select name="categoria" class="rounded-lg border-slate-300 text-sm">
    @foreach (['Todas', 'Tecnología', 'Cultura', 'Deporte'] as $cat)
      <option value="{{ $cat }}" @selected(request('categoria') === $cat)>
        {{ $cat }}
      </option>
    @endforeach
  </select>

  <label class="flex items-center gap-2 text-sm text-slate-600">
    <input type="checkbox" name="solo_destacados" value="1"
      @checked(request('solo_destacados')) class="rounded border-slate-300">
    Solo destacados
  </label>

  <button type="submit"
    class="rounded-lg bg-blue-600 px-4 py-2 text-sm font-medium text-white hover:bg-blue-700">
    Filtrar
  </button>
</form>

```

`@selected(cond)` imprime el atributo `selected` cuando la condición es verdadera; `@checked(cond)` hace lo propio con `checked`. Existen además `@disabled`, `@readonly` y `@required` con el mismo comportamiento.

Incluya el parcial al inicio de la cartelera, justo antes del listado:

```

resources/views/eventos/index.blade.php
Copiar

<x-layout title="Eventos">
  <h1 class="mb-6 text-2xl font-bold">Cartelera de eventos</h1>

  @include('partials.filtros')

  @foreach ($eventos as $evento)
    {{-- x-evento-card ... --}}
  @endforeach
</x-layout>

```

06 Problemas frecuentes

Síntoma	Causa probable	Solución
Cambié la vista pero no veo el cambio.	Vista compilada en caché o caché del navegador.	<code>php artisan view:clear</code> y recargue con <code>Ctrl+F5</code> .
El componente no se renderiza / "not found".	Nombre o ubicación incorrectos.	La etiqueta va en kebab-case (<code>evento-card</code>) y el archivo en <code>components/</code> .
El <code>class="mb-4"</code> que paso no se aplica.	El componente reescribe <code>class</code> fijo.	<code>{{ \$attributes->class([...]) }}</code> en la etiqueta raíz.
Undefined variable \$title.	La prop no está declarada.	Declárela en <code>@props</code> con un valor por defecto.
Los estilos de Tailwind no cargan.	Vite no está corriendo o el <code>@vite</code> falta.	<code>npm run dev</code> y verifique la línea <code>@vite</code> en <code><head></code> .

Síntoma	Causa probable	Solución
El slot con nombre sale vacío.	Sintaxis del slot.	Use <x-slot:nombre> y {{ \$nombre }} dentro del componente.
Se ve el texto {{ }} literal en la página.	El archivo no compila como Blade.	El nombre del archivo debe terminar en .blade.php.

07 Rúbrica de evaluación

Criterio	Pts
Ejercicio 1 — directivas de control, @forelse, @class y \$loop	25
Ejercicio 2 — layout reutilizable con componentes (navbar y footer extraídos)	25
Ejercicio 3 — componentes con props, slots con nombre y \$attributes (anónimo + de clase)	25
Ejercicio 4 — @include, directivas de formulario	25
Total	100

Referencias

Documentación oficial de Laravel 13 — Blade Templates. Documentación de Tailwind CSS v4.